

Assessment

What I would fix, in what order, and the risk of leaving each issue in place

The order, and why

Two findings are live exposures and close this week. Then observability and tests, because everything after them is a change to a system nobody can currently verify. The architecture — logic welded into route handlers — is the most visible problem and the least urgent one, so it goes last.

Four rules produced the order below:

1. Stop what is leaking data or money before fixing anything else.
2. Buy the ability to see and to verify before making substantial changes.
3. Harm to customers beats inconvenience to engineers.
4. Cheap and reversible jumps the queue.

No rewrite. The system works and earns money. A rewrite means a year of no product delivery to arrive back where we started, minus the bugs we forgot to reimplement.

Findings, in fix order

#	FINDING	RISK OF LEAVING IT	FIX BY
1	Browser reads the database directly, row-level security off	Any customer can read every customer's name, address and order history. Reportable breach	Week 1
2	Live Stripe and database credentials in the repo	Anyone with a clone can charge, refund, or read everything, silently	Week 1
3	No error tracking or alerting	We hear about outages from customers, hours late. Also blocks safe changes	Week 1
4	Deploys run from one laptop, no rollback	A bad release on a billing day has no fast undo	Week 1
5	No tests, no CI	Nothing above can be fixed safely. Blocks everything else	Month 1
6	Money paths have no transactions or idempotency	Wrong charges, manual reconciliation, revenue leakage	Month 1
7	One shared admin login	No attribution for refunds or data access. Insider risk	Month 1
8	Business logic inside route handlers	Pricing drifts across surfaces. Every change is slow and risky	Quarter 1
9	Node 16 (end of life), dependencies 3 years stale	Known CVEs, and we cannot adopt better tooling	Quarter 1

Findings 1 and 2 are both week 1. I would close 1 first: exploiting it needs a free account and the browser devtools, so the population who *could* is every customer we have. Finding 2 has higher impact per actor but a bounded population — fourteen people have ever had repo access.

The findings in detail

1. The storefront queries the database directly, with RLS off

The account pages use the Supabase client from the browser with the shared anon key. That is supported, but only with row-level security on, and it is on for none of the six tables that matter. Signed up as a customer, I read the full `customers` table from the browser console in four minutes.

Risk of leaving it: live exposure of every customer record we hold. If it has already been used we would not know — there is no query logging. Under UK GDPR this is a notifiable breach with a 72-hour clock that starts when we become aware.

Risk of the fix: enabling RLS blind breaks account pages, and broken account pages mean cancellations. So: log-only mode first, review a day of "would have been denied" queries, then enforce table by table starting with `customers`. Each table reverts on its own.

2. Live credentials are committed to the repository

`.env.production` was committed in 2021 and is still in the history: a live Stripe secret key, the Postgres connection string, and the session signing secret. The Stripe key still works.

Risk of leaving it: the payment key can create charges and refunds. The session secret can forge an admin session. The database password plus finding 1 is full read-write access. Nothing alerts on any of it. The exposure has been growing since 2021.

Risk of the fix: rotating the database password takes the site down if a consumer is missed, and I have already found two nobody remembered (a Metabase dashboard and a Zapier sync). So: inventory every consumer first, rotate Stripe using its dual-key window, rotate the database password in a low-traffic window with the old password held ready. This is the riskiest thing in week 1 and it gets a rehearsal and a written rollback.

3. We cannot see production

No error tracking, no alerting, no dashboards. Of nine incidents since January, seven were first reported by a customer, and the median time to notice was just over three hours.

Risk of leaving it: every incident lasts longer than it needs to. More importantly it blocks the rest of this list — I am about to change a system that cannot go down, and the current signal for "did that break something" is a customer complaint.

Risk of the fix: almost none. Additive, touches no business logic, removable. That is why it is in week 1 despite not being an exposure.

4. Deploys run from one laptop and there is no way back

`git pull && npm run build && pm2 restart` over SSH, run by one person. No staging. Rolling back means reverting and repeating it — 25 minutes last time.

Risk of leaving it: a bad deploy during billing week, when a broken checkout loses orders that do not come back, and the one person who can deploy may not be available.

5. There are no tests and no CI

Zero test files in six years. That is what happens when a small team ships continuously and the code's shape makes tests genuinely hard to write.

Risk of leaving it: no direct customer harm today, but it multiplies everything else. Every fix above is a change nobody can verify. It is also why the team avoids the billing code, which is why the billing bugs in finding 6 have survived. A refactor without characterization tests is not a refactor, it is a rewrite with extra confidence.

6. The money paths have no transactions and no idempotency

The plan-change and checkout handlers both write to the database, call Stripe, then write again. No transaction, no idempotency key. If Stripe times out in between, the database and Stripe disagree and nothing detects it.

Support has 23 tickets tagged *charged wrong amount* in six months, each reconciled by hand against the Stripe dashboard at roughly 40 minutes a ticket.

Risk of leaving it: revenue leakage, refunds we should not owe, and a support burden that grows with the customer base. People cancel a coffee subscription over a wrong charge rather than complain.

This is the file rebuilt in the [worked refactor](#).

7. Everyone shares one admin login

One account, password in shared notes. Refunds, address changes and cancellations all go through it, and the audit log records `admin` as the actor.

Risk of leaving it: no administrative action can be attributed to a person. Offboarding is meaningless, insider risk is uninvestigable, and it fails the first serious diligence we face. Individual accounts are a day of work.

8. Business logic lives inside route handlers

Pricing appears in four places with three different rounding behaviours, and the mobile app reimplements it a fourth way. There is no single answer to what a deluxe box costs for a customer with a legacy discount and a promo code.

Risk of leaving it: almost pure velocity cost, which is why it is eighth. It makes pricing changes slow and frightening, guarantees web and mobile drift, and keeps the code untestable. It is the largest body of work here and the least urgent.

9. Node 16 and three-year-old dependencies

Node 16 left security support in September 2023. `npm audit` reports 47 advisories, 6 high severity in packages that touch request handling.

Risk of leaving it: most advisories are probably not exploitable in our configuration. The real cost is that being this far behind blocks the tooling that makes everything else cheaper, and the gap widens on its own.

If I could only do one thing

Enable row-level security. It is the only finding where the harm is unbounded, ongoing, and invisible to us.

The [migration plan](#) sets out what ships in week 1, month 1 and quarter 1.